

Deploy Runbook — stand up the Indigenomics RAP Portal from scratch

Consolidated, linear “zero → running” checklist. Companion to the deeper `deploy.md` and `deploy-rap.md`, and the handoff context in `PROJECT-AUDIT.md`.

This is the single ordered path to deploy the portal into an AWS account — **including a fresh, client-owned account**. Follow the sections top to bottom; the order matters (e.g. the `AuthSecret` must exist *before* the first deploy). No credentials appear in this document — every secret is a **placeholder you supply**, and `<YOUR_ACCOUNT_ID>` stands in for the target AWS account.

What you’re deploying: a Next.js 14 app on SST v4 + OpenNext — the app runs as a Lambda behind CloudFront, talking to DynamoDB. One `sst deploy` creates the CloudFront distribution, the server + image Lambdas, all DynamoDB tables, and the S3 buckets. `sst.config.ts` (repo root) is the source of truth.

0. Decide before you start

Decision	Options	Notes
Stage name	<code>production</code> / <code>ca</code> / any dev name	<code>production</code> is <code>retain</code> (its data survives <code>sst remove</code>); all other stages are <code>remove</code> . See the caveat below.
Region	<code>us-east-1</code> (default) / <code>ca-central-1</code> (residency)	See §3. Bedrock/BDA/Texttract availability differs by region.
Extraction engine	<code>mock</code> / <code>bda</code> / <code>bedrock</code>	<code>mock</code> first to validate infra, then turn on real AI (§6).
Legal-cases data	receive an export / rebuild via pipeline	The <code>LegalCases</code> table is not SST-managed — see §5.2.

⚠️ **Only `production` retains data — `ca` does not.** The removal policy keys on the literal stage name `production` (`sst.config.ts:27`): `production` is `retain`, and **every other stage — including `ca` — is `remove`**, so `sst remove` (or deleting the stack) destroys that stage’s DynamoDB tables and S3 buckets and all their data. This matters because `ca` is the Canadian-residency stage. If you run a **real** residency environment there, protect it deliberately — enable DynamoDB point-in-time recovery + backups, or move the retention rule off the hardcoded `"production"` name — **before** it holds real data.

1. Prerequisites — per machine (one-time)

- [] **Node 20** installed (`node -v`). (The repo has no `engines` pin; 20 is what CI uses.)
- [] **AWS CLI v2** + credentials for the target account — SSO or `aws configure`.
- [] `git clone` the repo and `npm install` (this brings in SST v4).
- [] **Docker** only if you want the local DynamoDB loop (`npm run ddb:up`) — not needed to deploy.

- [] Confirm you're pointed at the right account: `aws sts get-caller-identity` → shows `<YOUR_ACCOUNT_ID>`.

SST version note: `package.json` pins `sst@4.15.2`. Some in-repo comments still say "SST v3 (Ion)"; before the first deploy, sanity-check the `sst.config.ts` shapes flagged inline (`transform` PITR, `stream`, `subscribe`, `bucket cors`) against the installed version. See the version-drift note in `PROJECT-AUDIT.md` §5.

2. One-time account setup (per AWS account) — the handoff-critical part

Do these **once per account**, before the first deploy. This is the part that differs when moving from the sandbox to a client-owned account.

2.1 Session secret (required — deploy fails without it)

```
npx sst secret set AuthSecret "<A_RANDOM_32+_BYTE_STRING>" --stage <stage>
```

Use a freshly generated random value (e.g. `openssl rand -base64 32`). Stored in SSM Parameter Store by SST — **never commit it**. Set it for **each stage** you deploy.

2.2 Bedrock model access (required for real extraction)

In the **Bedrock console** → **Model access**, in your chosen `BEDROCK_REGION`, enable access to:

- [] A **Claude** model (RAP extraction — `InvokeModel` 403s without it).
- [] **Amazon Titan Text Embeddings V2** (`amazon.titan-embed-text-v2:0`) — embeddings.
- [] **Llama 3.3 70B** (legal-cases briefs / Q&A), if using the cases features.

2.3 SES sender identity (required for notifications)

- [] In the stage's region, **verify** your `DIGEST_SENDER` address as an SES identity.
- [] While the account is in the SES **sandbox**, also verify `DIGEST_RECIPIENT` (sandbox SES only sends to verified addresses). Request production SES access to lift this.

2.4 BDA project (required only for the `bda` engine / `production` stage)

The default `production` stage references a BDA project by ARN. **In a new account those ARNs won't resolve** — recreate the project and override the ARNs (see §6, Option A). If you only use the `bedrock` or `mock` engine, skip this.

2.5 CI deploy role (optional)

Manual `sst deploy` needs none of this. To enable push-to-`main` auto-deploy, create a **GitHub OIDC provider + deploy role** in `<YOUR_ACCOUNT_ID>` and set the repo secret `AWS_DEPLOY_ROLE_ARN`. Until then, deploy manually — it's the supported path.

2.6 Budget alarm (recommended)

Create an AWS Budget alarm on day one. **Cost is almost entirely Bedrock LLM inference** and is easy to run up during bulk extraction — see `PROJECT-AUDIT.md` §3.

3. Region & residency strategy

- **No residency requirement** → deploy everything in one region (`us-east-1` is the default and where BDA/Texttract/Titan/Llama all work). Simplest.
- **Canadian data residency** → deploy the app + tables in `ca-central-1`, but note **BDA and Texttract are not fully available there**; Bedrock reaches Claude via the `us.` inference profile (data at rest in Canada, inference geo-routes). The `ca` stage ships the in-country **text-layer** engine for exactly this reason.

The sandbox's Texttract/CloudTrail blocks are SCPs from the parent org, not the product. In a client-owned account with no such SCPs, the full Texttract OCR path becomes available.

4. Environment variables (set at deploy time)

SST reads these from the shell at deploy. For real deploys prefer **SST Secrets / SSM** over inline env where the value is sensitive.

Var	Purpose	Typical value
<code>SST_AWS_REGION</code>	region to deploy into	<code>us-east-1</code> or <code>ca-central-1</code>
<code>EXTRACTION_IMPL</code>	extraction engine	<code>mock</code> / <code>bda</code> / <code>bedrock</code>
<code>DOC_LOADER</code>	document loader	<code>texttract</code> / <code>textlayer</code>
<code>BEDROCK_REGION</code>	Bedrock/BDA/Texttract region	<code>us-east-1</code>
<code>BEDROCK_MODEL_ID</code>	Claude inference-profile id (bedrock engine)	e.g. <code>us.anthropic.claude-...</code>
<code>BDA_PROJECT_ARN</code>	your BDA project (bda engine)	<code>arn:aws:bedrock:us-east-1:<YOUR_ACCOUNT_ID>:data-automation-project/...</code>
<code>BDA_PROFILE_ARN</code>	data-automation profile (bda engine)	<code>arn:aws:bedrock:us-east-1:<YOUR_ACCOUNT_ID>:data-automation-profile/us.data-automation-v1</code>
<code>REVIEW_MODE</code>	<code>indigenomics</code> (queue) / <code>off</code> (auto-publish)	<code>indigenomics</code>
<code>RAP_CORS_ORIGINS</code>	allowed upload origins (comma-sep)	your CloudFront/custom-domain URL + <code>http://localhost:3000</code>
<code>DIGEST_SENDER</code> / <code>DIGEST_RECIPIENT</code>	notification email (verified SES)	your addresses
<code>ALERTS_EMAIL</code>	observability SNS subscription (ca stage)	your address
<code>WAF_BLOCKING</code>	flip the CloudFront WAF from count-only to blocking (<code>observe</code> stages)	unset = count-only; <code>true</code> = enforce
<code>RAP_TABLE</code> / <code>RAP_UPLOAD_BUCKET</code> / <code>RAP_ANALYTICS_BUCKET</code>	wired by SST	(<i>auto</i>)

The `ca` stage bundles its vars in the npm script (§5.1) so you don't set them by hand.

5. Deploy

5.1 First deploy (mock engine — validate infra)

```
# default region / us-east-1
npx sst deploy --stage <stage>
```

```
# ca-central-1 residency stage (vars are baked into the script — don't hand-roll this one)
npm run ca:deploy
```

⚠ For the `ca` stage always use `npm run ca:deploy` / `npm run ca:diff`. Hand-rolling `sst deploy --stage ca` silently drops `SST_AWS_REGION`, `EXTRACTION_IMPL`, and `DOC_LOADER`, degrading the deploy.

Note the outputs: the **CloudFront URL** and the SST-generated **table/bucket names**.

5.2 Seed / data bootstrap

- [] **Portal demo data:** `npx sst shell --stage <stage> -- tsx scripts/seed-sst.ts` (resolves the per-stage table names automatically). This is an **upsert** of the canonical fixtures — safe to re-run.
- [] **Legal-cases corpus (`LegalCases`)** — *not SST-managed*. Choose one:
- **Receive an export** from the outgoing team and restore it, **or**
- **Rebuild via the pipeline** (`cases:create:cloud`, `cases:ingest:cloud`, `cases:harvest-*:cloud`, `cases:embed:bedrock:cloud`, `cases:index-build:cloud` — the `cases:*` scripts in `package.json`). The harvesters scrape external court sites; run them deliberately.

Do **not** run `ddb:create:cloud` / `rap:create:cloud` against SST-managed stages — they target bare table names and would create a second, wrong table. Always seed against the SST-generated name.

5.3 Before-handoff data hygiene (new client production)

- [] Delete the 103 `@demo` company accounts and retire the shared `demo-portal-2026` password. **Keep** `institute@demo` (the Indigenomics staff account — a separate singleton) for the Institute's initial access, but give it a real, private password. Optionally leave one or two demo company logins for exploring. Run `npx sst shell --stage <stage> -- tsx scripts/purge-demo-logins.ts` (dry-run; add `--apply` to delete, `--keep a@demo,b@demo` to retain extras) — it keeps `institute@demo` by default.
- [] Review `src/lib/commitments/org-bn-map.ts` (self-flagged "VERIFY BEFORE PROD MIGRATION").
- [] Replace the sample fixture corpora in `scripts/fixtures/` with the client's own documents.

6. Turn on real extraction

Deploy once with `mock` to prove the infra, then switch the engine.

Which engine? See the empirical comparison — `./03-rap-engine-comparison.md` (live n=8 run across three engines). Recommended for a client-owned, unrestricted account: Option B with `DOC_LOADER=extract` (Bedrock + Texttract-LAYOUT) — best coverage, real (read, not inferred) page numbers, and the only engine that processed all 8 test docs. Use `DOC_LOADER=textlayer` as the cheapest / best-grounding / most residency-friendly fallback for born-digital PDFs (no OCR, so it can't read scanned docs). Use BDA (Option A) only where speed matters more than provenance. Note: the LLM extraction step is Bedrock inference either way, and Canada is not a Bedrock inference geography, so inference leaves Canada regardless of engine ("in-CA" = data-at-rest + OCR in Canada).

Option A — BDA (managed, multi-page, us-east-1 only)

1. Create the blueprint from `src/lib/rap/bda-blueprint.json` in us-east-1: `aws bedrock-data-automation create-blueprint --type DOCUMENT --blueprint-stage LIVE --schema file:///... --region us-east-1`
2. Create a Data Automation project referencing it; note the **project ARN**.
3. Redeploy with `EXTRACTION_IMPL=bda`, `BEDROCK_REGION=us-east-1`, `BDA_PROJECT_ARN=<your new project ARN>`, `BDA_PROFILE_ARN=arn:aws:bedrock:us-east-1:<YOUR_ACCOUNT_ID>:data-automation-profile/us.data-automation-v1`.

Option B — Claude on Bedrock (fully in-country, e.g. ca-central-1)

Keeps processing in Canada and grounds every field in a verbatim quote.

```
SST_AWS_REGION=ca-central-1 EXTRACTION_IMPL=bedrock BEDROCK_MODEL_ID=<Claude profile id> \
npx sst deploy --stage <stage>
```

Requires `texttract:StartDocumentTextDetection` + `GetDocumentTextDetection` + `bedrock:InvokeModel`. (In the sandbox these Texttract calls are SCP-blocked, which is why the `ca` stage uses `DOC_LOADER=textlayer` instead — not needed in an unrestricted account.)

7. Verify the deploy

`URL=<the CloudFront URL from step 5.1>`

```
curl -s -o /dev/null -w "%{http_code}\n" "$URL/coverage" # expect 200
```

- [] Open the URL and walk **report** → **confirm** → **coverage** → **Index**.
 - [] Sign in as **Indigenomics** → `/rap/upload`, upload a sample RAP → job reaches `PENDING_REVIEW` (flagged) or auto-publishes → appears on `/rap`.
 - [] Record a progress update → the `RollupAggregator` recomputes (check its CloudWatch logs).
 - [] **ca stage only**: confirm the SNS **email subscription** (click the confirmation link sent to `ALERTS_EMAIL`) so alarms can notify.
 - [] **ca** + **production** (**observe**): a WAFv2 WebACL auto-attaches to the CloudFront distribution (rate-limit + AWS managed CommonRuleSet + KnownBadInputs), created in us-east-1. It ships **count-first** — nothing is blocked yet. Confirm attachment: `aws cloudfront get-distribution-config --id <id>` shows a non-empty `WebACLId`. After watching `AWS/WAFV2` count-mode metrics for false positives, redeploy with `WAF_BLOCKING=true` to enforce. No SCP change was needed for the deploy role.
-

8. Cost & teardown

- Serverless infra ≈ free at demo scale; **real cost is Bedrock/BDA/Texttract per page/token**. See `PROJECT-AUDIT.md` §3 for real figures.
- **Tear a stage down when idle:** `npx sst remove --stage <stage>`.
- Remember `production` is `retain` — its data buckets survive a remove and need manual cleanup if you truly want everything gone.
- **Non-production stages (including `ca`) do NOT retain** — `sst remove` deletes their DynamoDB tables and S3 buckets and all their data. Back up first if the stage holds anything you need (see the §0 caveat).

9. Troubleshooting

Symptom	Fix
Token has expired / <code>ExpiredToken</code>	Refresh creds (<code>aws sso login</code> or re-auth).
<code>sst deploy</code> fails complaining about a secret	<code>AuthSecret</code> not set for this stage — do §2.1.
<code>InvokeModel</code> <code>AccessDenied</code> / 403	Bedrock model access not enabled in <code>BEDROCK_REGION</code> — §2.2.
BDA job <code>ClientError</code>	Wrong blueprint field names / project ARN, or missing <code>BDA_PROFILE_ARN</code> .
Presigned upload blocked (CORS)	Set <code>RAP_CORS_ORIGINS</code> to your app origin; confirm the bucket <code>cors</code> block deployed.
App shows no data after deploy	Tables exist but weren't seeded — run §5.2.
Seed wrote to the wrong/empty table	Pass/resolve the SST-generated table name, not a literal like <code>RapData</code> .
Rollup never updates	Streams enabled on <code>RapData</code> ? <code>RollupAggregator</code> IAM / <code>RAP_TABLE</code> env? Check its logs.
<code>AccessDenied</code> on a service that should work	You may be under an org SCP (as the sandbox is for Texttract/CloudTrail) — confirm with the account owner.
<code>sst deploy</code> type/prop error	Reconcile the flagged <code>sst.config.ts</code> API shapes with the installed SST version (§1).

No credentials or real secret values appear in this runbook — only placeholders you supply. `<YOUR_ACCOUNT_ID>` is the target AWS account.